

CodeAlpha Cyber Security Internship

Task 1: Basic Network Sniffer

Submitted By: Abdullah Saeed

Domain: Cyber Security

Organization: CodeAlpha

STUDENT ID: CA/DF1/197436

1. Introduction

A Network Sniffer is a cybersecurity tool used to capture and monitor network packets transmitted across a network. This project was developed using Python and the Scapy library to analyze live network traffic.

2. Objective

- Capture live network packets.
- Display Source IP Address.
- Display Destination IP Address.
- Detect protocol (TCP/UDP/ICMP).
- Display packet length.
- Display payload when available.

3. Tools and Technologies

- Python 3
- Scapy
- Npcap
- Visual Studio Code
- Windows

4. Project Features

- Live packet capture
- Source and Destination IP detection
- Protocol identification

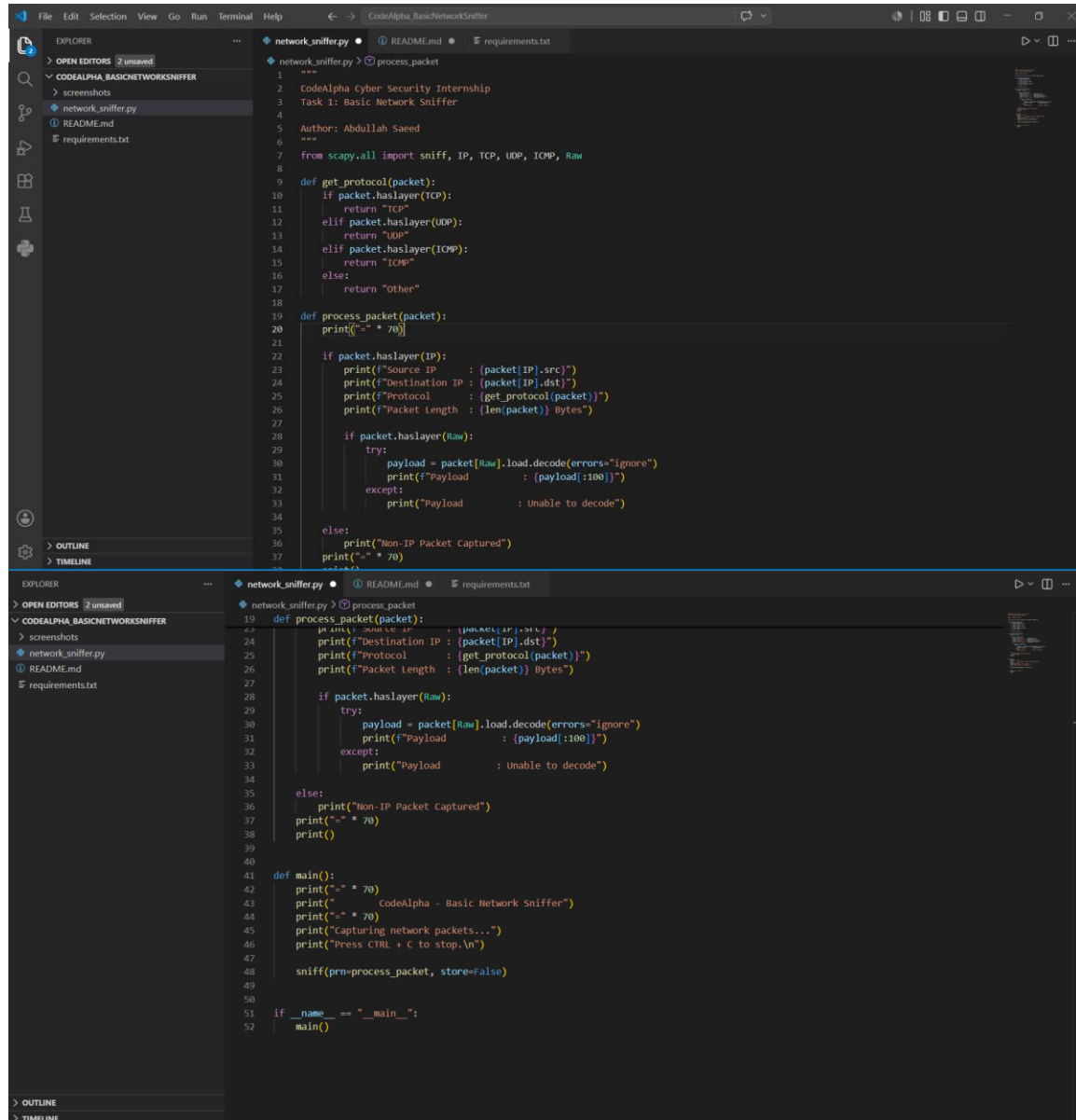
- Packet length display
- Payload display

5. Working of the Program

The application uses Scapy's sniff() function to continuously monitor network traffic. For each captured IP packet, it extracts the source IP, destination IP, protocol, packet length, and payload (when available), then prints the information in the terminal until the user stops the program with Ctrl + C.

6. Screenshots

• Source Code Screenshot



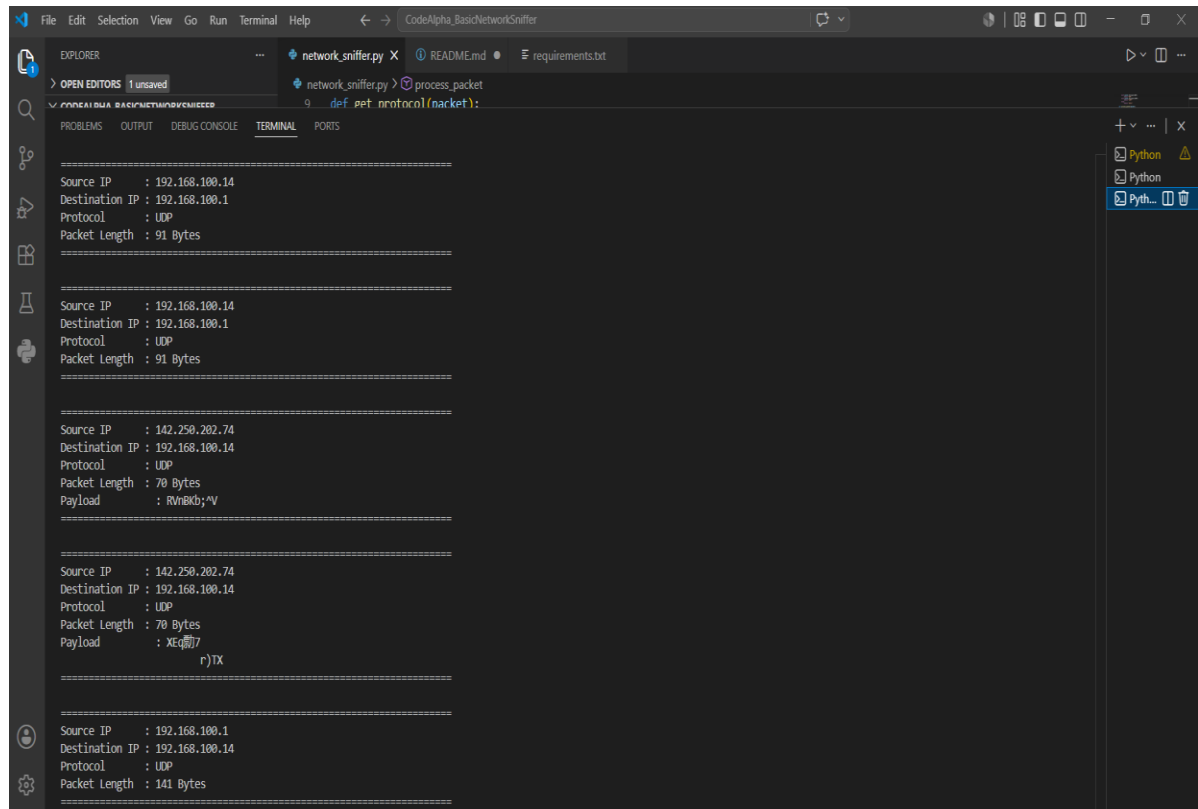
The image displays two screenshots of a Visual Studio Code editor window, showing the source code of a network sniffer script named `network_sniffer.py`. The editor is configured with a dark theme and has the Explorer sidebar on the left, showing the project structure: `CODEALPHA_BASICNETWORKSNIFFER`, `screenshots`, `network_sniffer.py`, `README.md`, and `requirements.txt`.

The top screenshot shows the `process_packet` function, which handles incoming packets. It includes a `get_protocol` helper function that identifies the protocol based on the packet's layer (TCP, UDP, ICMP, or Other). The `process_packet` function prints the source and destination IP addresses, the protocol, and the packet length. It also attempts to decode the raw payload, displaying it if successful or an error message if it fails.

The bottom screenshot shows the `main` function, which sets up the network sniffing process. It prints a header for the script, captures network packets using `sniff`, and processes each packet using the `process_packet` function. The script is designed to run as a standalone program or as a module.

```
1 """
2 CodeAlpha Cyber Security Internship
3 Task 1: Basic Network Sniffer
4
5 Author: Abdullah Saeed
6 """
7 from scapy.all import sniff, IP, TCP, UDP, ICMP, Raw
8
9 def get_protocol(packet):
10     if packet.haslayer(TCP):
11         return "TCP"
12     elif packet.haslayer(UDP):
13         return "UDP"
14     elif packet.haslayer(ICMP):
15         return "ICMP"
16     else:
17         return "Other"
18
19 def process_packet(packet):
20     print("-" * 70)
21
22     if packet.haslayer(IP):
23         print(f"Source IP : {packet[IP].src}")
24         print(f"Destination IP : {packet[IP].dst}")
25         print(f"Protocol : {get_protocol(packet)}")
26         print(f"Packet Length : {len(packet)} Bytes")
27
28     if packet.haslayer(Raw):
29         try:
30             payload = packet[Raw].load.decode(errors="ignore")
31             print(f"Payload : {payload[:100]}")
32         except:
33             print("Payload : Unable to decode")
34
35     else:
36         print("Non-IP Packet Captured")
37     print("-" * 70)
38
39
40
41 def main():
42     print("-" * 70)
43     print("CodeAlpha - Basic Network Sniffer")
44     print("-" * 70)
45     print("Capturing network packets...")
46     print("Press CTRL + C to stop.\n")
47
48     sniff(prn=process_packet, store=False)
49
50
51 if __name__ == "__main__":
52     main()
```

• Program Output Screenshot



The screenshot shows a Visual Studio Code window with a terminal running a network sniffer program. The terminal output displays five captured network packets with their details. The Explorer sidebar on the left shows the project structure with files like `network_sniffer.py`, `README.md`, and `requirements.txt`. The bottom right shows the Python extension's file explorer.

```
Source IP      : 192.168.100.14
Destination IP : 192.168.100.1
Protocol       : UDP
Packet Length  : 91 Bytes

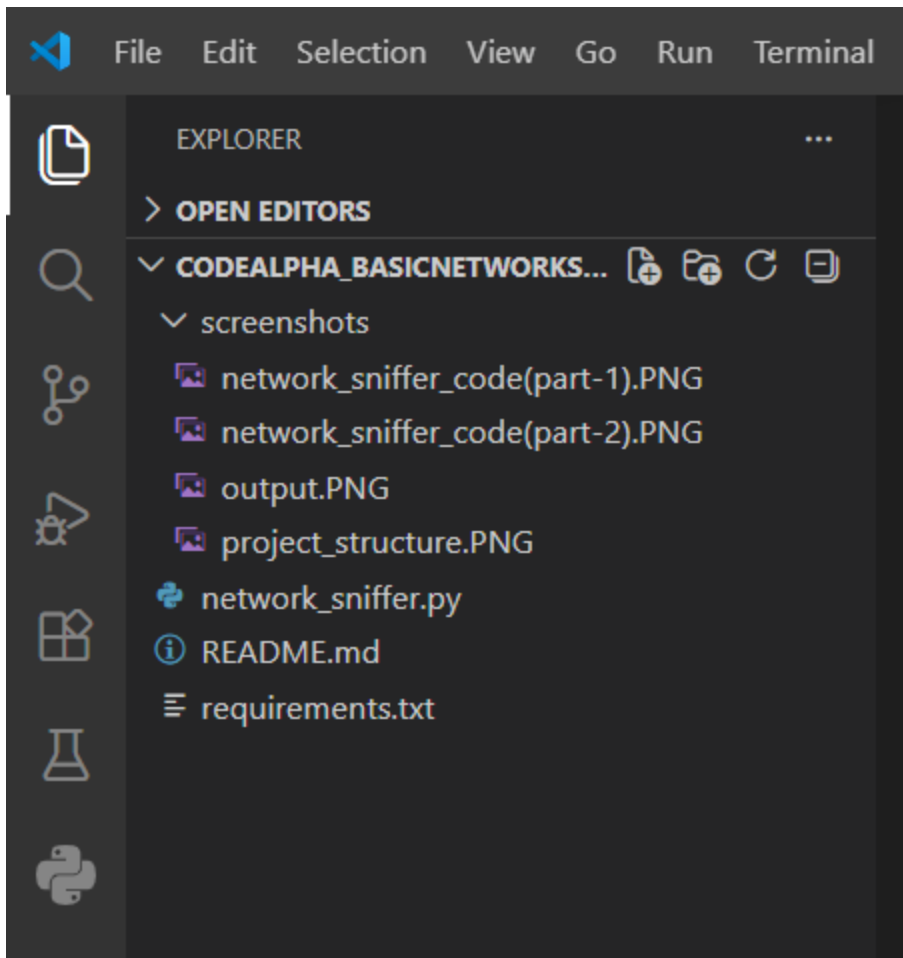
Source IP      : 192.168.100.14
Destination IP : 192.168.100.1
Protocol       : UDP
Packet Length  : 91 Bytes

Source IP      : 142.250.202.74
Destination IP : 192.168.100.14
Protocol       : UDP
Packet Length  : 70 Bytes
Payload        : RvNBKh;^V

Source IP      : 142.250.202.74
Destination IP : 192.168.100.14
Protocol       : UDP
Packet Length  : 70 Bytes
Payload        : Xi-q前7
                    r)TX

Source IP      : 192.168.100.1
Destination IP : 192.168.100.14
Protocol       : UDP
Packet Length  : 141 Bytes
```

• Project Structure Screenshot



7. Results

The Basic Network Sniffer successfully captured live network packets and displayed useful information including source IP, destination IP, protocol, packet length, and payload.

8. Conclusion

This project provided practical experience in packet capturing and network traffic analysis using Python and Scapy. It successfully fulfilled the objectives of the internship task.

9. Future Improvements

- GUI
- Packet filtering

- Save packets to file
- PCAP export
- Traffic statistics

10. References

Python: <https://www.python.org/>

Scapy: <https://scapy.readthedocs.io/>

Npcap: <https://npcap.com/>